



LUNDS
UNIVERSITET

FMN050 VT06: Numerical analysis for E/I
Numerisk Analys, Matematikcentrum

Project 2

Last hand-in date: **19 May 2006 at 13:15**

Aim: To study the solution of differential equations in an applied problem.

Grading: Accounts for 10p of the course's total 50p (5p required for passing grade).

Project reports: *The project must be solved and reported individually.* You are encouraged to discuss the problems with your fellow students, but you must construct your own MATLAB programs yourself, run them, interpret the results, and report on your findings in your own report to be graded.

Notes: Make sure that your report has your name, "personnummer," section and your class year, e.g. E03. In your report you must acknowledge all help you received, e.g. from discussions with fellow students or with an instructor. Finally, you should end your report by summing up what you have learned from solving the problems. *Reports must be handed in printed on paper, with accompanying graphics and figures. No electronic submission can be accepted.*

The projects are part of the examination, and your approved and graded project reports remain valid for one year. Read all instructions on the course web site.

In this computer assignment, the goal is to learn how to use a two point boundary value solver in MATLAB. The code you will use is a fourth order method, `bvp4c`, and you will apply it to a biomathematical problem that simulates nerve impulse propagation.

In the central nervous system, nerve impulses are propagated in thin strands called axons. The impulse propagation is a complex electro-chemical process. Seydel (1988) suggests the following simple differential equation to model nerve impulse propagation:

$$\begin{aligned}\dot{y}_1 &= 3(y_1 + y_2 - \frac{y_1^3}{3} - 1.3) \\ \dot{y}_2 &= -\frac{1}{3}(y_1 - 0.7 + 0.8y_2).\end{aligned}$$

Here the dot denotes derivative with respect to time t . This is an initial value problem, but the impulse propagation is periodic in time, with unknown

period T . This is expressed in terms of *boundary conditions* $y_1(0) = y_1(T) = 0$, $y_2(0) = y_2(T)$, which specify that the solution is *periodic*. This makes our differential equation a *boundary value problem*, as the solution at the endpoint must coincide with the initial value.

A difficulty is that the period T is unknown. The following is a common trick to rewrite such a problem as a standard boundary value problem. Change the independent variable from t to $x = t/T$. Then we see that $x = 0$ corresponds to $t = 0$ while $x = 1$ corresponds to $t = T$. The unknown endpoint $t = T$ has been transformed to a known endpoint, $x = 1$. The ODE becomes

$$\begin{aligned} y_1' &= 3T(y_1 + y_2 - \frac{y_1^3}{3} - 1.3) \\ y_2' &= -\frac{T}{3}(y_1 - 0.7 + 0.8y_2) \end{aligned}$$

subject to the boundary conditions

$$y_1(0) = 0; \quad y_1(1) = 0; \quad y_2(0) = y_2(1).$$

Here prime denotes derivative with respect to x . **The task is to solve this problem using `bvp4c`.**

The problem is nonlinear. This means that we will have to use iterations to solve the problem. Therefore the code will require an “initial guess” for the solution. As the solution is periodic on $[0, 1]$, we choose the initial guess $y_1(x) \approx \sin(2\pi x)$ and $y_2(x) \approx \cos(2\pi x)$. (This corresponds to the guess $T = 2\pi$.) We note that this guess satisfies the boundary conditions.

In order to solve the problem, you must write a function that describes the differential equation. Hence you need to implement a

```
function dydx = ode(x,y,T)
```

that represents the differential equation’s right-hand side, and a function for the initial solution guess,

```
function v = guess(x).
```

Both are vector functions, i.e., the output argument `dydx` and `v` are both 2-vectors. The latter is a vector function with two components containing the initial guess on $[0, 1]$ for the two solution components, y_1 and y_2 .

Because the solver will have to iterate to solve this nonlinear problem, you finally have to implement a function that evaluates the *residual*, i.e., how well the guess (or further approximations) satisfies the equations. This is a

```
function residual = bcdeviation(yleft,yright,T)
```

where `yleft` is the value of the solution at the left endpoint of the interval, i.e., $(y_1(0), y_2(0))$, while `yright` is the corresponding value at the right endpoint. The function `bcdeviation` calculates the three deviations

1) `yleft(1)` (equal to $y_1(0)$ and representing this boundary condition's deviation from the correct value 0; the problem is solved when `yleft(1)=0.`);

2) `yright(1)` (equal to $y_1(1)$ and representing this boundary condition's deviation from the correct value 0; the problem is solved when `yright(1)=0.`);

3) `yleft(2)-yright(2)` (This equals $y_2(0) - y_2(1)$; as the solution is supposed to be periodic, this deviation is zero only when a periodic solution has been found; the problem is solved when `yleft(2)-yright(2)=0.`)

The three deviations `yleft(1)`, `yright(1)` and `yleft(2)-yright(2)` are placed as components 1, 2 and 3, respectively, in the function's output vector `residual`, which therefore becomes a 3-vector.

How to present your results.

After you have computed the solution, you need to present your results. The following is a list of plots that should be considered a minimum.

1) Plot the final solution as well as the initial guess as a function of time. Note that the program will find T for you, so you need to rescale the plot accordingly. Thus, when you have found T , you reintroduce the original independent variable $t = xT$, so that you can plot the solution vs. t . It is suggested that you plot the solution over *two* periods, so that one can clearly see that the solution is periodic.

2) In a periodic problem of this type, it is common to plot y_2 as a function of y_1 . Because the problem is periodic, this will form a closed loop in the (y_1, y_2) plane, provided that you plot these variables over a full period. Produce such a plot!

Tips, hints and suggestions.

Often it is a bit tricky to solve a nonlinear BVP. In this problem a lot of things can go wrong. In most cases this happens because your initial guess is poor, but there are also other reasons. The most common reasons are

- if the period of the initial guess is too far off the exact, but unknown period T
- if the amplitude of the periodic initial guess is too large
- if you suggest an initial guess that is incompatible with the boundary conditions, i.e., a guess that doesn't satisfy the boundary conditions
- if you suggest too few grid points to start with

It may then happen that the code doesn't converge. As you don't get any solution at all in that case, it is very obvious that the code failed. However, in some situations you may obtain a "false" solution – one that is quite

far away from the true solution but nevertheless satisfies the differential equations within some tolerance.

In order to convince yourself that you have indeed obtained a real solution to the problem, we advise you to take the following steps:

- Extensive testing with different initial guesses and parameters is always a good practice
- You can use the **options** provided by **bvp4c**. There you can affect the computational accuracy (via the **tolerance**)
- Once you “know” what the solution is it is often a good idea to try to find out if you solve the problem efficiently. This is a matter of how many grid points you need, and how many calls to the function **ode** that you needed to compute the solution
- To be able to “control” the code and the solution process, it is very important to learn how it works. Read the **help** file, and try some of the possibilities suggested there